9.2 gRPC - современный RPC фреймворк

Что такое gRPC?

Определение:

gRPC — открытый RPC фреймворк от Google (2016)

- **g** = Google, **RPC** = Remote Procedure Call
- Основан на **HTTP/2** и **Protocol Buffers**

Универсальный механизм для обмена данными между сервисами

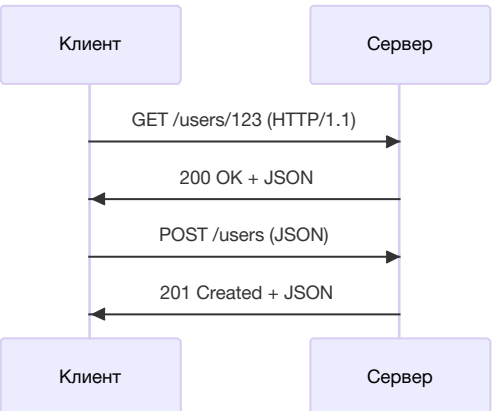
Ключевые особенности:

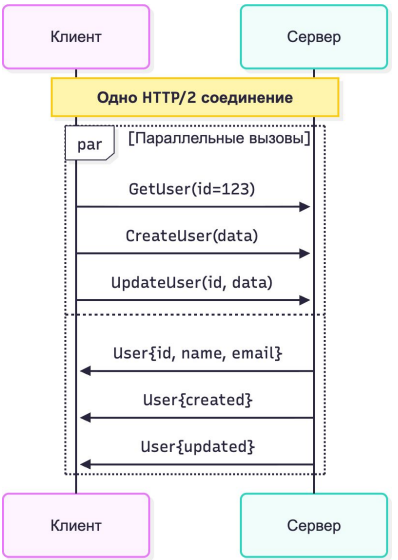
- **Высокая производительность** — бинарная сериализация
- **Мультиязычность** — поддержка 10+ языков
- **Строгие контракты** — схема данных в .proto файлах
- **Потоковая передача** — 4 типа взаимодействия



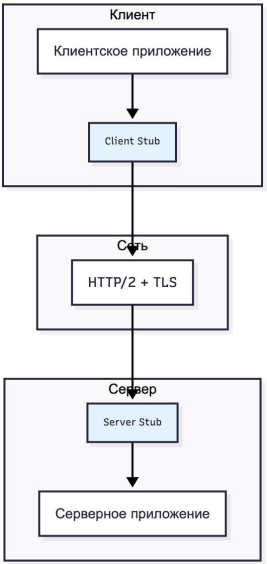
Идея: вызывать удаленные функции как локальные!

Традиционный REST:




Клиент
Сервер
Одно HTTP/2 соединение
par
[Параллельные вызовы]
GetUser(id=123)
CreateUser(data)
UpdateUser(id, data)
User{id, name, email}
User{created}
User{updated}
Клиент
Сервер

Архитектура gRPC



Процесс работы:

1. **Определение контракта** в .proto файле
2. **Генерация кода** для клиента и сервера
3. **Реализация логики** на выбранном языке
4. **Вызов методов** как локальных функций

Protocol Buffers

Что это такое?

Protocol Buffers (ProtoBuf) — язык описания данных от Google

- **Схема данных** в .proto файлах
- **Бинарная сериализация** — быстро и компактно
- **Автогенерация кода** для разных языков
- **Обратная совместимость** при изменениях

Пример .proto файла:

```
syntax = "proto3";

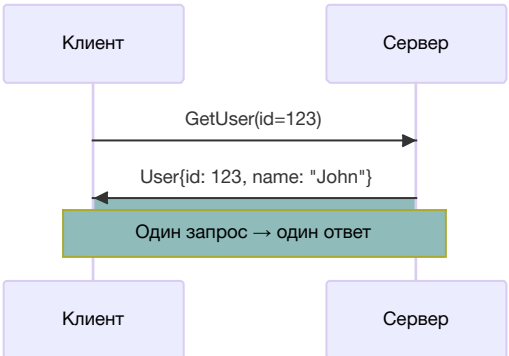
package user;

// Определение структуры пользователя message
User {
  int32 id = 1; string name = 2;
  string email = 3;
  repeated string roles = 4; // массив строк
}

// Запрос получения пользователя message
GetUserRequest {
  int32 user_id = 1;
}
// Ответ с данными пользователя message
GetUserResponse {
  User user = 1;
}
// Определение сервиса
service UserService {
  rpc GetUser(GetUserRequest) returns (GetUserResponse);
}
```

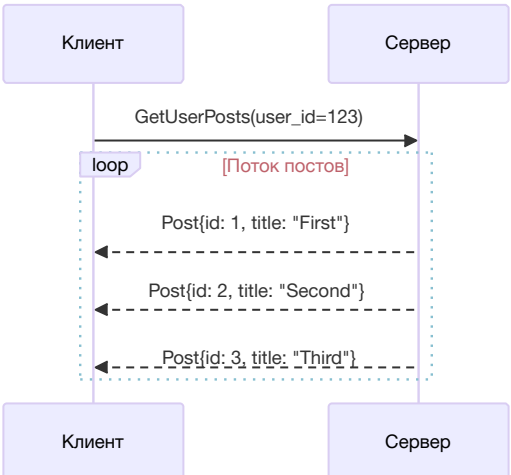
Типы взаимодействия в gRPC

1. Unary RPC — простой запрос-ответ

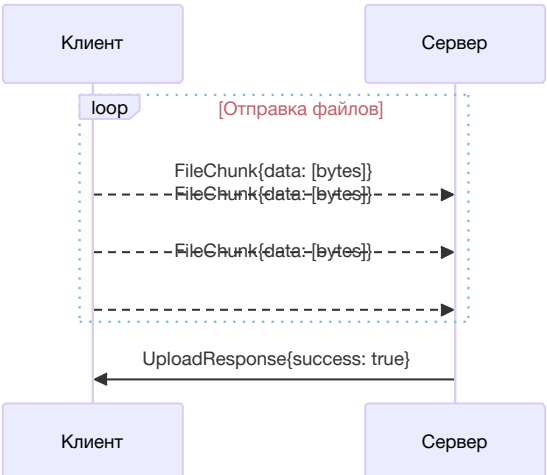


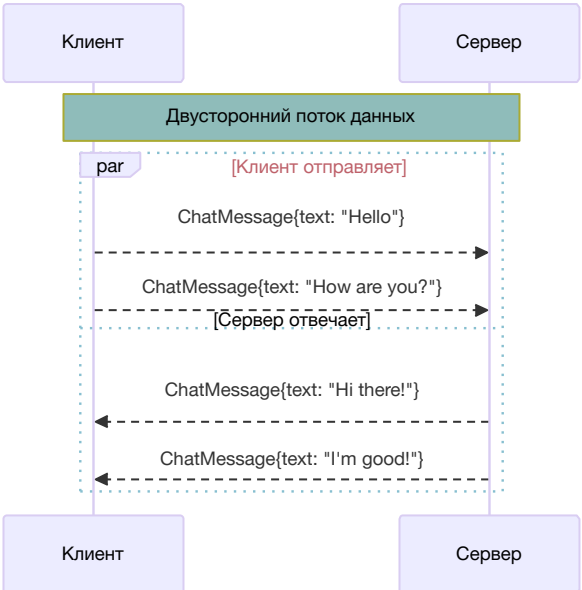

```
service UserService {
  rpc GetUser(GetUserRequest)
  returns (GetUserResponse);
}
```

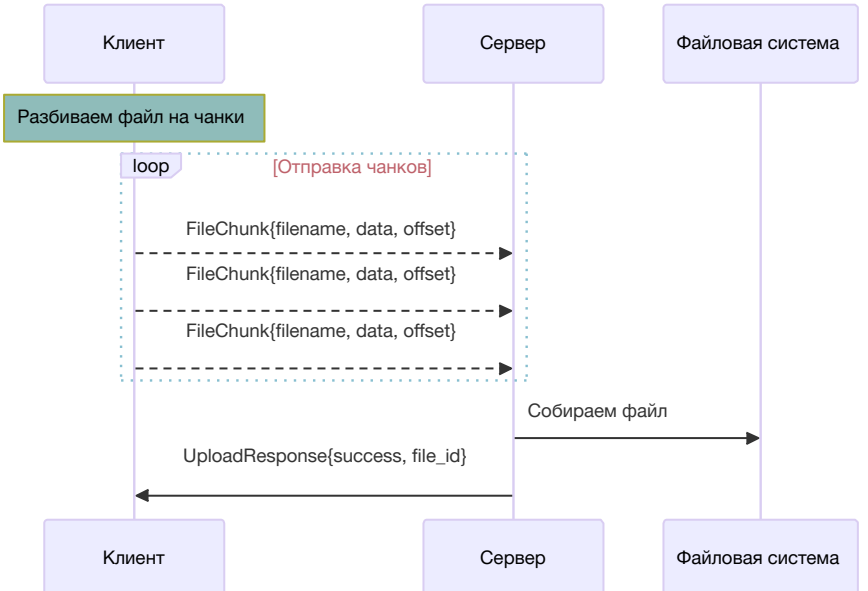
2. Server Streaming — поток от сервера



3. Client Streaming — поток от клиента




Клиент
Сервер
Двусторонний поток данных
par
[Клиент отправляет]
ChatMessage{text: "Hello"}
ChatMessage{text: "How are you?"}
[Сервер отвечает]
ChatMessage{text: "Hi there!"}
ChatMessage{text: "I'm good!"}
Клиент
Сервер


Клиент
Сервер
Файловая система
Разбиваем файл на чанки
loop
[Отправка чанков]
FileChunk{filename, data, offset}
FileChunk{filename, data, offset}
FileChunk{filename, data, offset}
Собираем файл
UploadResponse{success, file_id}
Клиент
Сервер
Файловая система

Сценарии использования:

gRPC подходит для:

- **Микросервисная архитектура** — внутренние API
- **Высокие нагрузки** — низкая задержка
- **Потоковые данные** — real-time коммуникация
- **Мобильные приложения** — экономия трафика

gRPC НЕ подходит для:

- **Простые CRUD операции** — избыточная сложность
- **Веб-браузеры** — ограниченная поддержка
- **Небольшие команды** — высокий порог входа

Заключение

Ключевые преимущества gRPC:

- **Производительность** — до 10x быстрее REST
- **Компактность** — 60% экономии трафика
- **Стриминг** — real-time коммуникация
- **Типобезопасность** — строгие контракты
- **Мультиязычность** — 10+ языков программирования